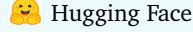




Safin-1: Safety from Within through Memory-Native State Evolution

Safin Team, Shanghai AI Laboratory



Long-horizon complex tasks require foundation models to accumulate information, maintain internal states, and adapt over extended interactions. In these settings, safety should be an intrinsic property of the model itself, rather than a behavioral constraint that depends solely on external safeguards or post-hoc alignment procedures such as supervised fine-tuning. This motivates *Safety from Within*, in which safety-relevant capabilities are represented and invoked through the model’s native computation rather than relying solely on external safeguards. We present **Safin-1**, a family of foundation models that realizes this principle through memory routing and state evolution. Safin-1 is built on Memory-Anchor Routing across Context History (MARCH), a novel network architecture that maintains structured memory states and selectively retrieves relevant historical information through content-conditioned routing. Safin-1 further supports test-time adaptation of persistent capability states without repeatedly modifying the backbone model, enabling controlled specialization over a shared general-purpose foundation. We investigate this persistent-memory interface on downstream safety tasks through a *Safety State*, demonstrating effective state-based adaptation with substantial safety improvements. More broadly, the routed-state interface unifies contextual memory and persistent capability adaptation within the model’s native computation. This reframes model memory from a passive record of prior context into an active substrate for maintaining and evolving model behavior over time. Evaluations across general capabilities, long-context understanding, retrieval, and efficiency further validate Safin-1. Together, these findings provide a concrete path toward safety as a state-native and adaptively maintainable capability. We emphasize that this work represents only an initial architectural exploration of *Safety from Within*, and substantial further work is needed to realize this broader vision.

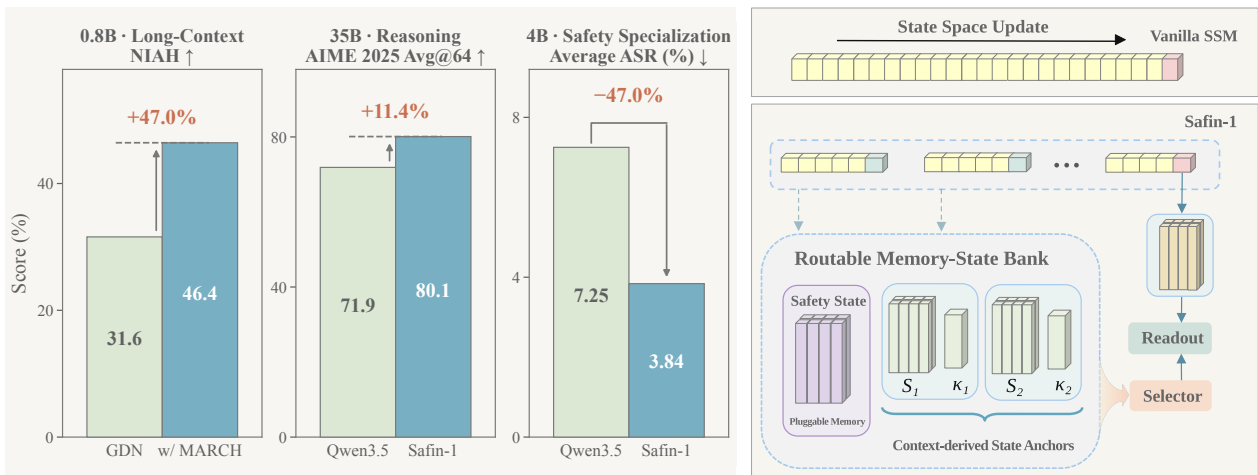


Figure 1 | Safin-1 architecture and representative results. Left: results from 0.8B architectural validation and 35B-A3B scaling. The 4B safety panel compares Qwen3.5 with Safin-1 equipped with its learned Safety State. Right: the Safin-1 architecture routes context-derived and persistent memory states for selective readout.



Contents

1	Introduction	3
2	Technical Methodology	4
2.1	Context-Derived State Anchors	5
2.2	Content-Routed State Retrieval	6
2.3	Persistent Capability States	7
2.4	Efficient Producer–Reader Implementation	7
3	Small-Scale Validation of MARCH	8
3.1	Experimental Setup	8
3.2	Language Modeling and Long-Context Capabilities	8
3.2.1	General Language Modeling	8
3.2.2	Length Extrapolation and Associative Recall	9
3.2.3	Long-Context Understanding and In-Context Retrieval	9
3.3	Architecture Analysis	10
3.4	Efficiency and Scalability	12
4	Safin-1 at Scale	13
4.1	Continual Pretraining and Supervised Fine-Tuning	13
4.1.1	Experimental Setup	13
4.1.2	Capability and Safety Evaluation	14
4.2	Safety from Within via a Persistent Safety State	15
4.2.1	Training Data and Experimental Setup	16
4.2.2	Safety–Utility Trade-off	16
5	Related Works	18
6	Discussion and Conclusion	19
	Project Contributors	20
	References	21
A	Training Configurations	30
A.1	Small-Scale MARCH Validation	30
A.2	Large-Scale Safin-1 Training	31
A.3	Safety State Specialization	32

1. Introduction

Long-horizon intelligence requires two forms of continuity. A model must retain what has happened across an extended context, and it must reliably invoke the capabilities appropriate to the current situation. Language models typically realize these functions through different substrates: context is retained in a token-level key-value cache or a recurrent state, whereas reusable behaviors such as safety are commonly encoded through model updates or adapters, or imposed at inference time through learned prefixes and representation interventions [1, 2, 3, 4]. At a broader level, the R²AI perspective [5] argues that scalable safety should evolve alongside model capability [6, 7], combining resistance to known threats with resilience to unforeseen risks [8, 9, 10, 11, 12]. These observations motivate a broader question: can model state serve not only as a transient compression of context, but also as a native substrate for persistent and selectively invoked capabilities? We study this question through memory and safety, two settings that require a model to recover relevant historical information while invoking appropriate behavioral constraints without indiscriminately suppressing useful responses [13].

Self-attention preserves fine-grained access to prior tokens but incurs quadratic training cost and a key-value cache that grows with sequence length [14]. Recurrent and linear-attention models instead compress the causal prefix into a fixed-size state, enabling efficient constant-memory decoding [15, 16]. Selective state-space models and delta-rule recurrences substantially improve how this state is written, revised, and forgotten [17, 18, 19]. However, conventional recurrent designs expose only the latest state for direct reading. Once an earlier association is attenuated or overwritten, its previous representation may no longer be recoverable from the current state alone [20, 21]. Recent work alleviates this *single-state bottleneck* by enlarging recurrent memory, routing among multiple states, or retaining temporally organized state collections [22, 23, 24, 25, 26, 27]. Yet retaining additional states introduces a second challenge: historical states must be represented compactly and retrieved according to the current context. A scalable recurrent memory therefore requires not only state preservation, but also an efficient state-addressing interface.

We present **Safin-1**, a family of foundation models designed to explore this state-native formulation of long-horizon intelligence and safety. Safin-1 is built on **Memory-Anchor Routing across Context History (MARCH)** [28], which preserves earlier versions of the model’s evolving recurrent state as addressable internal memory. Beyond context-derived memories, the same routing interface can host learned persistent capability states. We instantiate this interface as a detachable *Safety State*, allowing Safin-1 to acquire safety specialization through test-time adaptation of model state while keeping the shared language-model backbone frozen.

At the architectural level, the recurrent backbone periodically checkpoints its evolving state to form addressable *state anchors*. Each anchor is paired with a compact, state-aware routing key, allowing every token to retrieve relevant historical states or select a learned null option when historical memory is unnecessary. Retrieved states are fused with the current recurrent path, preserving the behavior of the underlying recurrence. This design retains the model’s own state trajectory as causal, end-to-end trainable memory native to the recurrent computation.

This addressable state bank also provides Safin-1 with an interface for persistent capability specialization. The bank can contain both dynamic anchors derived from the current context and learned persistent states that encode reusable specialization. We investigate safety as the first instance of the latter. With the language-model backbone frozen, we learn a persistent *Safety State* and place its layer-wise states in the same bank as the context-derived anchors; the existing router then determines their contribution for each token. The memory can be attached or removed without rewriting the shared backbone. Learned from harmful and benign examples [29], Safety State places safety specialization inside the model’s native recurrent-state system while preserving a shared general-purpose

foundation. We view this state-native and selectively invoked safety capability as a concrete realization of *Safety from Within*, also referred to as *Making Safe AI* [5, 30, 31, 32].

We evaluate Safin-1 and its underlying architecture through two complementary stages. First, controlled 0.8B studies isolate the architectural foundation of Safin-1 by pretraining matched models on 50B tokens and applying MARCH to Gated DeltaNet, Kimi Delta Attention, and Gated DeltaNet-2 [19, 33, 34]. These experiments isolate the contribution of the proposed memory-routing architecture and establish consistent improvements in general language modeling, LongBench [35], real-world in-context retrieval, and RULER NIAH [36], including extrapolation beyond the training context.

Second, we scale Safin-1 to 4B and 35B-A3B using the hybrid Qwen3.5 backbones [37] through matched continual pretraining and supervised fine-tuning. The resulting variants raise the macro-average over ten capability benchmarks from 66.79 to 69.20 and from 76.25 to 78.35, respectively. The gains are most pronounced on challenging reasoning and competition-level mathematics. Together, these results suggest that state routing strengthens the model’s ability to sustain and recover useful computation over long reasoning horizons without sacrificing broad general-purpose capability. The same pattern across the dense 4B and mixture-of-experts 35B-A3B configurations further indicates that these benefits persist across model scales and backbone organizations.

Within the scaled Safin-1 study, we additionally start from the Safin-1 SFT checkpoints, freeze the language-model backbone, and investigate the effectiveness of a persistent Safety State. This state-native specialization reduces average jailbreak attack success rate by 42.3% at 4B and 52.3% at 35B-A3B, while producing substantially less over-refusal than a training-matched rank-8 LoRA control [38]. This study therefore represents an initial architecture-level exploration of *Safety from Within*, opening a broader research agenda for alternative and complementary architectures that make safety native to model computation.

We additionally develop fused dense and sparse routing implementations and evaluate their efficiency up to 128K tokens. Sparse state routing provides a state-level analogue of sparse attention: it selects among compressed recurrent memories rather than token-level keys, making historical retrieval practical at long sequence lengths [39, 40].

Our main contributions are summarized as follows:

- **Safin-1 and its state-native architecture.** We introduce Safin-1, a family of foundation models built on MARCH. Its architecture converts the transient trajectory of recurrent computation into a growing bank of addressable state anchors, enabling selective access to earlier model states without altering the underlying recurrent update.
- **Persistent capability states for Safety from Within.** We extend the memory-state bank with learned persistent capability states and instantiate this interface as a detachable Safety State. This mechanism specializes a frozen Safin-1 backbone through its native state-routing pathway, making safety a model-native and selectively invoked capability.
- **Controlled and scaled empirical validation.** We validate the underlying memory-routing architecture through controlled 0.8B pretraining studies across multiple recurrent backbones, then scale Safin-1 to 4B and 35B-A3B through continual pretraining and supervised fine-tuning. Across general capability, long-context, retrieval, safety, and efficiency evaluations, the results demonstrate consistent architectural gains and a favorable safety-over-refusal trade-off.

2. Technical Methodology

Safin-1 augments recurrent computation with a unified, routable bank of model-internal states. Its state-routing backbone is instantiated by MARCH and comprises three components: context-derived state anchors that preserve earlier versions of the evolving recurrent state, token-conditioned routing

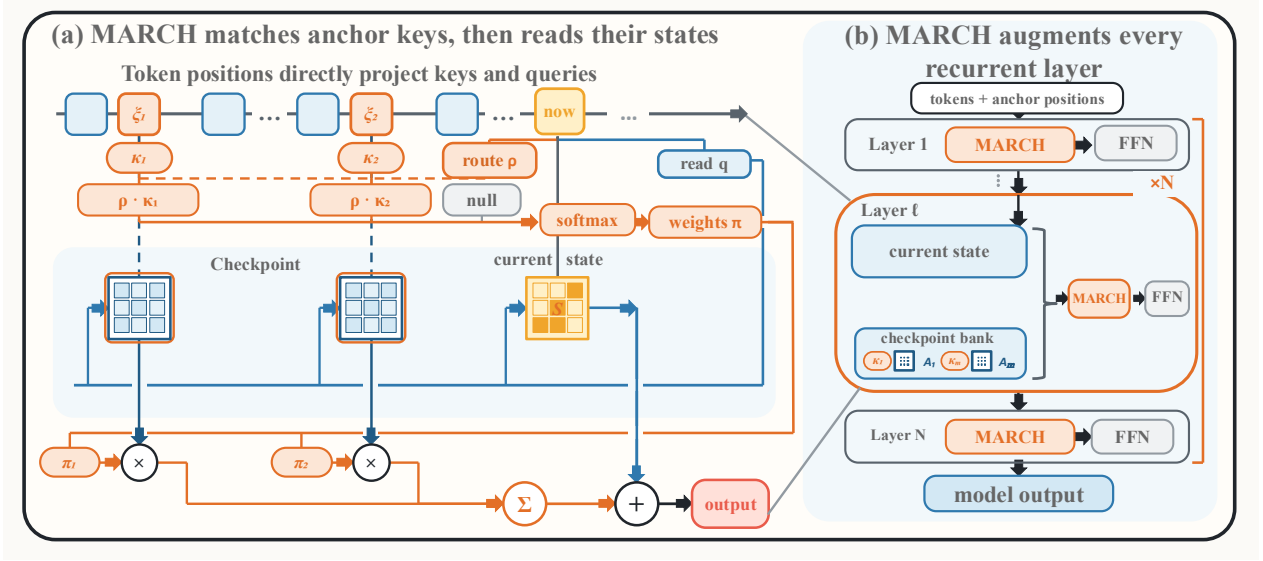


Figure 2 | MARCH state-routing backbone underlying Safin-1. Left: the continuously evolving recurrent state is periodically checkpointed into addressable state anchors. Each token routes over causally visible anchor keys and a learned null option, and the retrieved historical readout is fused with the current-state path. Right: the state-routing mechanism is applied at every recurrent layer, with state-aware routing keys reconstructed across layers.

that selects relevant states or a learned null option, and persistent capability states that remain available independently of the current sequence. For each token, the selected state readout is fused residually with the current-state path, preserving the underlying recurrent update. Figure 2 illustrates the dynamic anchoring and historical-reading pathway; persistent capability states are introduced in Section 2.3.

2.1. Context-Derived State Anchors

Anchor placement. Let $\mathcal{T} = [t_1, \dots, t_L]$ be a sequence of L text tokens. An anchoring policy specifies an ordered set of text boundaries $\mathcal{B} = \{b_m\}_{m=1}^M$, where $0 = b_0 < b_1 < \dots < b_M \leq L$. We insert one occurrence of a shared learned anchor embedding after each boundary:

$$\widehat{\mathcal{T}} = \big\|_{m=1}^M \left([t_{b_{m-1}+1}, \dots, t_{b_m}] \parallel [\xi_m] \right) \parallel [t_{b_M+1}, \dots, t_L]. \quad (1)$$

where $\parallel$ denotes sequence concatenation, and ξ_m is the m -th occurrence of the shared anchor embedding ξ . Text and anchor positions have distinct computational roles. Text positions apply the base recurrent update, allowing the state to evolve continuously across boundaries. Immediately after processing t_{b_m} , the architecture checkpoints the resulting cumulative state. The following anchor position ξ_m does not modify the recurrent state; instead, it reads the aligned checkpoint to construct compact routing metadata. Each boundary therefore produces a coupled pair: a recurrent-state snapshot and a representation through which that snapshot can later be addressed.

Cumulative recurrent-state checkpointing. At each state-bearing layer ℓ , the base recurrent update remains unchanged and carries the state $\mathbf{S}_t^{(\ell)} \in \mathbb{R}^{d_v \times d_k}$ continuously across anchor boundaries. At each boundary b_m , it snapshots the current state:

$$\mathbf{A}^{(m, \ell)} = \mathbf{S}_{b_m}^{(\ell)} \in \mathbb{R}^{d_v \times d_k}, \quad m = 1, \dots, M. \quad (2)$$

Because the recurrence is not reset at anchor boundaries, $\mathbf{A}^{(m,\ell)}$ represents the cumulative prefix up to position b_m , rather than only the most recent segment. We therefore refer to it as a *state anchor*. The ordered bank $\{\mathbf{A}^{(1,\ell)}, \dots, \mathbf{A}^{(M,\ell)}\}$ preserves the temporal trajectory of a single recurrent memory before later updates attenuate or overwrite earlier contents.

Content-conditioned anchor metadata. Let $\mathbf{u}_m^{(\ell)}$ denote the normalized input representation of anchor position ξ_m at layer ℓ . The anchor position reads only its aligned state checkpoint:

$$\mathbf{q}_m^{(\ell)} = \mathbf{W}_q^{(\ell)} \mathbf{u}_m^{(\ell)}, \quad \mathbf{o}_m^{(\ell)} = \mathbf{A}^{(m,\ell)} \mathbf{q}_m^{(\ell)}. \quad (3)$$

The same input representation is projected into a compact routing key:

$$\boldsymbol{\kappa}_m^{(\ell)} = \mathbf{W}_k^{(\ell)} \mathbf{u}_m^{(\ell)} \in \mathbb{R}^{d_r}. \quad (4)$$

The aligned readout is passed through the standard output projection and residual pathway, making the next-layer anchor representation $\mathbf{u}_m^{(\ell+1)}$ dependent on its associated checkpoint $\mathbf{A}^{(m,\ell)}$. Consequently, although all anchor positions share the same input embedding, their representations—and therefore their routing keys—become state-dependent after the first layer. The router thus addresses anchors according to their retained contents rather than their temporal indices alone.

2.2. Content-Routed State Retrieval

Candidate scoring and null routing. For a text token at position t , the causally available state anchors are indexed by $\mathcal{V}_t = \{m \in \{1, \dots, M\} \mid b_m < t\}$. In the dynamic-memory setting, these anchors form the non-null candidate set $C_t = \mathcal{V}_t$. The routing module projects the normalized hidden state $\mathbf{x}_t$ into a routing query and scores it against the key of each candidate:

$$\boldsymbol{\rho}_t = \mathbf{W}_R \mathbf{x}_t, \quad a_{t,j} = \boldsymbol{\rho}_t^\top \boldsymbol{\kappa}_j, \quad j \in C_t. \quad (5)$$

To allow the model to bypass state retrieval, we augment the candidate set with a null option $\emptyset$, whose payload is fixed to zero, $\mathbf{A}^{(\emptyset)} = \mathbf{0}$. Its query-dependent logit is $n_t = \mathbf{w}_\emptyset^\top \mathbf{x}_t + b_\emptyset$. Let $\tilde{C}_t = C_t \cup \{\emptyset\}$ denote the augmented candidate set. We define the logit of each candidate $j \in \tilde{C}_t$ as

$$s_{t,j} = \begin{cases} a_{t,j}, & j \in C_t, \\ n_t, & j = \emptyset, \end{cases} \quad \pi_{t,j} = \frac{\exp(s_{t,j})}{\sum_{r \in \tilde{C}_t} \exp(s_{t,r})}. \quad (6)$$

The routing query $\boldsymbol{\rho}_t$ determines which candidates to retrieve, whereas the state-read query $\mathbf{q}_t$ reads their matrix-valued contents. Because the null candidate has a zero payload and participates in the same softmax, probability assigned to it attenuates the retrieval branch without introducing a separate fusion gate. If C_t is empty, the null candidate receives all probability mass.

Retrieved-state fusion. Given the routing probabilities, the candidate-state readouts are weighted and added to the current-state readout using the underlying recurrence’s native state-read query $\mathbf{q}_t$:

$$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t + \sum_{j \in \tilde{C}_t} \pi_{t,j} \mathbf{A}^{(j)} \mathbf{q}_t. \quad (7)$$

This additive formulation preserves the original recurrent path and introduces state retrieval as an auxiliary branch without modifying the underlying recurrent update. Because the routing probabilities

directly affect the layer output, the routing queries and state-derived keys are optimized end-to-end with the language-modeling objective.

We note that the aggregation formulation in Equation (6) readily admits a sparse variant by restricting aggregation to the K highest-scoring visible anchors (Top- K). This sparse approach exhibits natural connections with hierarchical sparse attention approaches [39, 41], while preserving dense token-level processing rather than relying on hard token-level pruning. Our ablation studies in Section 3.3 show that sparse routing substantially reduces aggregation cost with minimal performance degradation.

2.3. Persistent Capability States

Beyond context-derived anchors, the routed state bank at each state-bearing layer can also host J learnable persistent capability states, $\{\mathbf{P}_j\}_{j=1}^J$. We suppress the layer index in this subsection for clarity. Each persistent state has the same matrix-valued structure as a dynamic anchor, and its routing key κ_{p_j} is generated through the same state-aware metadata pathway defined in Equations (3)–(4). Unlike dynamic anchors, persistent states are learned parameters rather than checkpoints constructed from the current sequence. They are therefore available from the first text token without consuming positions in the input sequence.

We extend the non-null candidate set as

$$C_t = \mathcal{V}_t \cup \{p_1, \dots, p_J\}, \quad \mathbf{A}^{(p_j)} = \mathbf{P}_j, \quad (8)$$

where p_j identifies the j -th persistent candidate. Dynamic anchors and persistent states are scored by the same routing query and jointly normalized with the null option through a single softmax. Consequently, Equations (6) and (7) operate directly on the extended candidate set. The router therefore assigns each persistent state a token- and context-dependent contribution rather than adding it with a fixed weight at every position.

In Safin-1, we instantiate this interface for safety. The layer-wise collection $\{\mathbf{P}_{\text{safe}}^{(\ell)}\}_{\ell}$ constitutes the *Safety State*. During specialization, we keep the language-model backbone frozen and optimize only this collection; the training data and optimization setup are detailed in Section 4.2. Once learned, the Safety State can be attached to or removed from the routed state bank without changing the shared backbone parameters. This state-routed specialization provides the mechanism through which Safin-1 realizes *Safety from Within*.

2.4. Efficient Producer–Reader Implementation

The state-routing computation is organized into a recurrent producer and a fused state reader. The producer inherits the hardware-efficient chunkwise kernel of the underlying recurrent mixer. In the GDN-based implementation studied here, it follows the chunkwise formulation of Gated DeltaNet [19], processing recurrent updates in tensor-core-friendly blocks while producing the current-state readout and checkpointing the recurrent state at anchor boundaries. These checkpoints, together with any persistent capability states, form the candidates consumed by the state reader.

Following the I/O-aware principles of FlashAttention [42], the reader jointly tiles query tokens and state candidates, reuses each candidate tile across a block of queries, and fuses routing-score computation, normalization, and weighted state readout into a streaming reduction. This schedule avoids materializing either the dense token-to-candidate routing matrix or the substantially larger tensor of per-candidate matrix-valued readouts, reducing intermediate storage and HBM traffic.

The same producer supports both dense and Top- K retrieval. The sparse path leaves the recurrent computation and compact routing-score calculation unchanged, while restricting the more expensive matrix-valued state readout to the routed candidates. Safin-1 uses Top-4 routing by default. As shown

in Section 3.4, at 128K tokens, Top-4 routing more than doubles end-to-end throughput relative to dense routing and reduces core runtime by roughly an order of magnitude.

3. Small-Scale Validation of MARCH

We first isolate the architectural contribution of MARCH through controlled 0.8B pretraining studies across multiple recurrent backbones. By matching the training data, token budget, sequence length, and optimization protocol, these experiments test whether addressable historical states improve language modeling, long-context retrieval, and length extrapolation independently of model scale. The corresponding training configuration is provided in Appendix A.1.

3.1. Experimental Setup

Model Suite and Training Protocol. Our recurrent baselines include Gated DeltaNet (GDN) [19], GDN augmented with Log-Linear Attention [25], Kimi Delta Attention (KDA) [33], and Gated DeltaNet-2 (GDN2) [34]. We further apply MARCH to GDN, KDA, and GDN2 to evaluate its compatibility across different recurrent update rules. All recurrent models consist of 21 layers, with a hidden size of 1,536 and six value heads, totaling approximately 0.8B parameters. We also include two full-attention baselines for comparison: a depth-matched 21-layer model with 693M parameters and an approximately parameter-matched 24-layer model with 778M parameters. Both use 16 attention heads and a RoPE base of 500K. Following the academic-scale protocol of Log-Linear Attention, all models are pretrained from scratch on the same 50B-token subset of Long-Data-Collections, with a maximum sequence length of 16K. For MARCH, we set the routing dimension to $d_r = 64$ and place state anchors every $C = 512$ tokens. All other training configurations are held fixed across sequence mixers and summarized in Table 10.

Evaluation. We evaluate four complementary capability groups. *General language modeling* is assessed on eight zero-shot commonsense benchmarks: LAMBADA [43], PIQA [44], HellaSwag [45], WinoGrande [46], ARC-Easy and ARC-Challenge [47], OpenBookQA [48], and CommonsenseQA [49]. *Long-context understanding* is evaluated on LongBench [35]. For *controlled long-context retrieval and length extrapolation*, we evaluate three single-needle and three multi-needle NIAH tasks from RULER [36] at context lengths of 4K, 8K, 16K, 32K, and 64K. Since all models are pretrained with a maximum sequence length of 16K, performance at 32K and 64K measures zero-shot length extrapolation. Finally, *in-context retrieval* is evaluated on six real-world, recall-intensive datasets: SQuAD [50], TriviaQA [51], SWDE [52], FDA [53], Natural Questions [54], and DROP [55]. We follow the evaluation protocol of prior work [26] and use LM-Evaluation-Harness [56] where applicable.

3.2. Language Modeling and Long-Context Capabilities

3.2.1. General Language Modeling

Table 1 reports results on all eight zero-shot benchmarks. Among the recurrent models, MARCH achieves the best performance on every benchmark, raising the average score to 41.5, compared with 40.1 for Gated DeltaNet and 40.0 for its Log-Linear variant. Notably, MARCH also slightly outperforms both full-attention reference models on average. These results indicate that incorporating historical-state retrieval preserves general language modeling capability while improving access to long-range information. The improvement is broad rather than driven by a single outlier: MARCH exceeds the strongest recurrent baseline on each of the eight tasks, spanning both completion-oriented and discriminative commonsense evaluations. This consistency supports interpreting the aggregate gain as preservation of base capability rather than benchmark-specific specialization.

Table 1 | Zero-shot performance of MARCH and baseline models on eight commonsense reasoning benchmarks. Results are reported using accuracy (acc) or normalized accuracy (acc_n), as indicated in the column headers; higher is better ($\uparrow$). The best result among Gated DeltaNet variants in each column is highlighted in bold.

Model	LMB. acc $\uparrow$	PIQA acc $\uparrow$	Hella. acc $\uparrow$	Wino. acc $\uparrow$	ARC-e acc_n $\uparrow$	ARC-c acc_n $\uparrow$	OBQA acc_n $\uparrow$	CSQA acc $\uparrow$	Avg.
Transformer	49.4	66.5	33.9	52.1	47.8	26.4	32.0	22.1	41.3
w/ 24 Layers	50.3	67.6	34.4	50.6	46.3	25.8	31.2	24.7	41.4
Gated DeltaNet	48.5	66.1	33.1	50.8	45.9	25.3	30.0	21.1	40.1
w/ Log-Linear	47.7	65.7	33.2	51.9	44.3	24.9	30.4	21.7	40.0
w/ MARCH	49.5	66.9	34.8	52.6	47.1	25.6	32.8	22.5	41.5

3.2.2. Length Extrapolation and Associative Recall

We further evaluate associative recall using the NIAH suite from RULER [36]. Across the 24 task-length combinations in Figure 3, MARCH outperforms the strongest recurrent baseline in 19 settings and matches it in the remaining five. The gains are most pronounced on multi-needle tasks and at 32K, beyond the maximum training length of 16K. This behavior is consistent with the routing construction: because the same content router is shared across all state anchors, extending the sequence introduces additional retrievable states without requiring anchor-specific or length-specific parameters. The 32K results therefore suggest that the retrieval mechanism generalizes beyond the training context length, a property not captured by short-context performance alone.

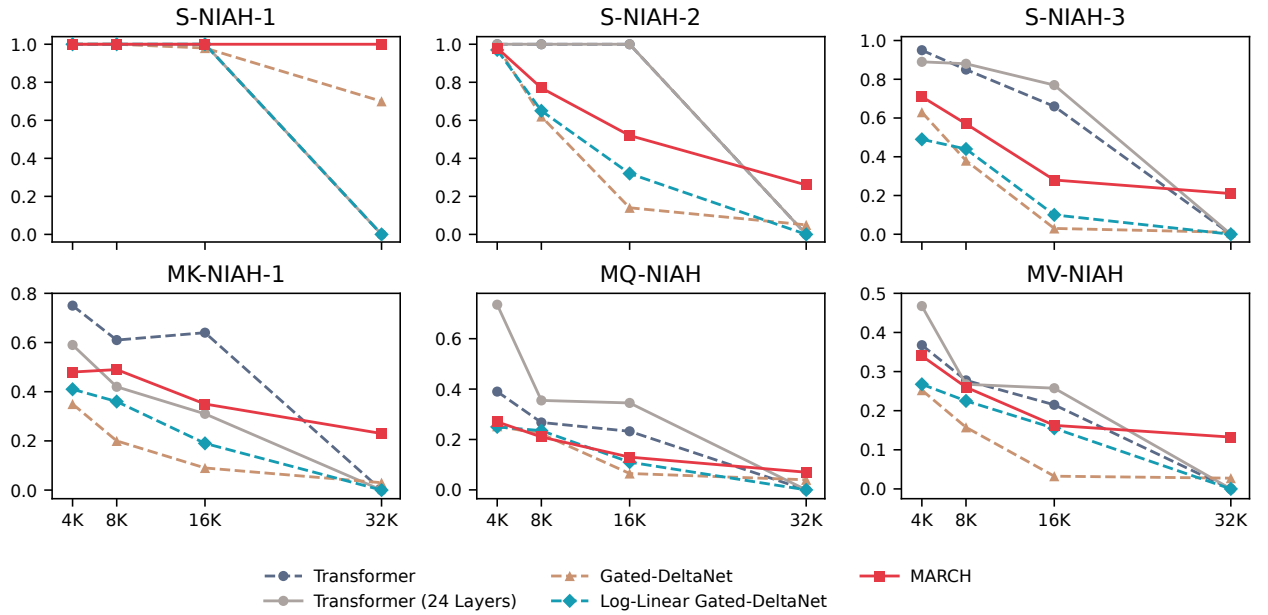


Figure 3 | NIAH performance on three single-needle and three multi-needle tasks. The Transformer achieves perfect accuracy on both S-NIAH-1 and S-NIAH-2 at context lengths of 4K, 8K, and 16K.

3.2.3. Long-Context Understanding and In-Context Retrieval

Table 2 presents results across all twelve LongBench tasks, covering single-document QA, multi-document QA, summarization, and few-shot learning.

Table 2 | Results on twelve LongBench tasks. The best result among Gated DeltaNet and its variants is shown in bold for each task. The relative average gain over the Gated DeltaNet is shown in green parentheses.

	Single-Doc QA			Multi-Doc QA			Summarization			Few-shot Learning			
Model	NQA	QQA	MFQ	HQA	2WM	Mus	GvR	QMS	MNs	TRC	TQA	SSM	Avg.↑
Transformer	4.4	4.1	15.9	7.5	9.9	4.1	10.7	11.6	14.5	21.0	33.9	28.3	13.8
w/ 24 Layers	3.5	11.1	18.3	7.8	9.8	4.1	11.3	12.9	12.8	22.5	47.5	23.2	15.4
Gated DeltaNet	3.0	4.8	13.3	5.6	8.7	2.1	2.6	11.3	13.0	18.0	38.6	21.9	11.9
w/ Log-Linear	3.6	6.2	13.6	7.1	8.2	3.3	6.3	13.2	13.5	17.0	32.8	25.1	12.5
w/ MARCH	4.2	7.8	14.6	7.4	11.5	4.8	8.2	17.4	14.1	19.0	43.1	26.3	14.9 (↑ 25%)

MARCH is the strongest recurrent model on every task, improving the overall average to 14.9, compared with 11.9 for Gated DeltaNet and 12.5 for Log-Linear Attention. The largest category-level gains occur in multi-document QA and summarization, where successful prediction requires integrating information distributed across distant portions of the context.

Table 3 | In-context retrieval accuracy (↑). The best result among Gated DeltaNet and its variants is marked in bold for each benchmark, with the relative gain over the stronger of the two Gated DeltaNet baselines shown in green parentheses.

Model	SQuAD↑	SWDE↑	FDA↑	TriviaQA↑	DROP↑	NQ↑	Avg.↑
Transformer	41.3	59.3	80.4	2.2	2.9	1.7	31.3
w/ 24 Layers	40.4	64.9	83.7	4.0	3.4	2.5	33.2
Gated DeltaNet	34.8	45.0	31.4	1.1	2.2	0.8	19.2
w/ Log-Linear	33.7	46.1	38.2	1.3	2.6	1.0	20.5
w/ MARCH	37.7 (↑ 8%)	51.9 (↑ 13%)	44.6 (↑ 17%)	1.6 (↑ 23%)	2.9 (↑ 12%)	1.2 (↑ 20%)	23.3 (↑ 14%)

Table 3 presents results on the six real-world in-context retrieval datasets. MARCH outperforms both recurrent baselines on every dataset, raising the best baseline average from 20.5 to 23.3. Together with the controlled RULER results, these improvements indicate that routed state anchors benefit retrieval beyond synthetic associative-recall settings, extending to heterogeneous real-world contexts.

3.3. Architecture Analysis

Generality across recurrent parameterizations. We further evaluate MARCH on GDN, KDA, and GDN2, which progressively increase the granularity and flexibility of recurrent memory updates: KDA refines GDN with channel-wise decay, while GDN2 further decouples erase and write operations. This progression allows us to examine whether the benefit of MARCH persists as the underlying recurrent update becomes more expressive. As shown in Table 4, MARCH improves NIAH in 11 of the 12 backbone-length settings. It also consistently improves SWDE and FDA, while largely preserving short-context performance and training perplexity. These results indicate that MARCH remains effective across recurrent updates with increasingly fine-grained memory control.

Positional encoding and length extrapolation. We study RoPE in hybrid models with a 3:1 recurrent-to-attention layer ratio, trained with a context length of 16K. Evaluations at 32K and 64K therefore directly measure length extrapolation. Table 5 shows that removing RoPE improves extrapolation for both MARCH and GDN and avoids the sharp degradation beyond the training context.

Table 4 | Comparison of GDN, KDA, and GDN2 with and without MARCH. NIAH scores are averaged over six needle-in-a-haystack tasks. The best result in each column is shown in bold.

Model	Train	Short-Context $\uparrow$		In-Context Retrieval $\uparrow$			NIAH (6-Task Avg.) $\uparrow$				
	PPL $\downarrow$	LAMB.	Hella.	SQuAD	SWDE	FDA	4K	8K	16K	32K	Avg.
GDN	6.796	48.52	33.10	34.85	45.00	31.40	53.80	35.67	21.73	15.13	31.58
w/ MARCH	6.751	49.52	34.84	37.70	51.85	44.56	59.58	54.96	39.46	31.71	46.43
KDA	6.691	52.11	35.41	38.64	51.94	37.93	51.17	34.75	22.54	18.08	31.64
w/ MARCH	6.703	52.57	34.95	33.58	54.10	42.83	58.96	52.25	32.13	22.17	41.38
GDN2	6.663	51.00	35.58	39.11	51.76	38.20	61.83	46.88	34.38	19.00	40.52
w/ MARCH	6.610	51.41	35.36	38.71	53.65	48.46	57.58	56.00	40.46	25.17	44.80

For GDN, however, this improvement comes at the cost of lower retrieval performance within 4K–16K. In contrast, MARCH benefits from removing RoPE across all context lengths, yielding both strong in-distribution retrieval and substantially smoother extrapolation to 32K and 64K.

Table 5 | Effect of RoPE in hybrid models trained up to 16K context. Results at 32K and 64K evaluate length extrapolation. The Transformer baseline is a parameter-matched 24-layer model. NIAH Avg. is averaged across the five context lengths. The best result in each column is shown in bold.

Model	RoPE	In-Context Retrieval $\uparrow$			NIAH (6-Task Avg.) $\uparrow$					
		SQuAD	SWDE	FDA	4K	8K	16K	32K	64K	Avg.
MARCH	✓	40.82	58.42	78.58	76.88	72.29	66.88	13.79	4.13	46.79
	×	41.12	65.44	80.76	87.79	79.92	74.54	68.79	30.25	68.26
GDN	✓	43.40	58.42	76.77	79.96	73.58	65.79	13.29	9.08	48.34
	×	40.11	65.17	75.23	68.00	61.04	55.63	48.46	21.33	50.89
Transformer	✓	40.48	64.99	83.76	78.04	65.38	66.88	0.00	0.00	42.06

Effect of the anchor interval. The anchor interval C controls the temporal resolution of the state bank: shorter intervals yield denser historical anchors, but increase storage and routing overhead. We vary C from 256 to 2,048 while keeping all other model and training settings fixed. Table 6 reports performance on three in-context retrieval benchmarks and the average over six NIAH tasks at 4K, 8K, and 16K context lengths. We further evaluate whether a model trained with $C = 512$ can operate at different state-bank resolutions at inference time, including the Fenwick-tree organization adopted by Log-Linear Attention [25, 57].

When training and inference resolutions are matched, $C = 512$ provides the best overall trade-off. Increasing C reduces the temporal resolution of the state bank and generally degrades retrieval, whereas further decreasing it to $C = 256$ yields little consistent benefit despite introducing more anchors. Notably, the $C = 512$ checkpoint can use a denser state bank at inference time, with $C = 256$ achieving the strongest aggregate performance. Performance deteriorates as the bank becomes increasingly sparse, while the Fenwick-tree variant remains competitive despite using a state organization unseen during training, indicating that the learned historical-state reader transfers across different state-bank layouts.

These results also expose a practical deployment control: the same checkpoint can trade retrieval quality against state-storage and routing cost by changing only the inference-time bank construction.

Table 6 | Anchor-interval ablation. Panel (a) matches training and inference; panel (b) varies inference for the $C = 512$ checkpoint. NIAH averages six tasks per length; column-wise best results are bold.

Anchor Interval C	# Anchors at 16K	In-Context Retrieval $\uparrow$			NIAH (6-Task Avg.) $\uparrow$		
		SQuAD	SWDE	FDA	4K	8K	16K
(a) Matched training and inference anchor intervals							
256	64	36.76	49.23	44.83	58.17	49.25	44.83
512	32	37.70	51.85	44.56	59.58	54.96	39.46
1024	16	36.49	48.51	43.28	53.21	43.21	33.54
2048	8	34.15	44.19	30.76	59.54	39.83	27.96
(b) Inference-time anchor interval							
64	256	39.41	53.11	46.01	63.33	55.00	39.67
128	128	39.08	52.84	46.91	63.33	57.33	39.17
256	64	40.35	53.38	47.46	62.83	56.17	40.83
512	32	37.70	51.85	44.56	59.58	54.96	39.46
1024	16	37.23	43.74	34.57	55.33	48.33	32.67
2048	8	37.23	33.93	23.23	51.00	40.33	34.00
Fenwick Tree	6	36.16	36.79	22.35	52.17	39.56	33.14

Routing design. We ablate three components of the routing mechanism: the query-key dimension d_r , routing sparsity, and the learned null option. Table 7 reports macro-averaged results on general language understanding, LongBench, and in-context retrieval, together with NIAH performance. Our default configuration uses dense routing with $d_r = 64$ and includes the null option.

Table 7 | Routing-design ablations. The first row is the default; each subsequent row changes one component. Commonsense (CS), LongBench, and Retrieval are macro-averages over 8, 12, and 6 benchmarks, respectively. NIAH scores are averaged over six tasks, and Avg. over the three context lengths. Column-wise best results are bold.

Configuration			Benchmark Averages $\uparrow$			NIAH (6-Task Avg.) $\uparrow$			
Routing	d_r	Null	CS	LongBench	Retrieval	4K	8K	16K	Avg.
Dense (default)	64	Yes	41.48	14.87	23.31	59.58	54.96	39.46	51.33
Dense	192	Yes	40.94	13.88	24.52	56.71	51.33	37.38	48.47
Top-4	64	Yes	41.38	13.79	23.17	57.29	46.04	31.21	44.85
Dense	64	No	41.04	14.11	22.86	54.88	47.92	35.13	45.98

Increasing d_r from 64 to 192 improves in-context retrieval but degrades the other aggregate metrics, making $d_r = 64$ a better overall trade-off. Top-4 routing largely preserves general language understanding and retrieval performance, but underperforms dense routing on NIAH, suggesting a viable efficiency-accuracy trade-off rather than a stronger default. Finally, removing the null option consistently degrades performance across all aggregates, indicating that explicitly allowing the router to ignore irrelevant historical states is important for robust routing.

3.4. Efficiency and Scalability

Training efficiency. Figure 4 compares end-to-end throughput and core forward-backward runtime for FlashAttention-2, Gated DeltaNet, dense MARCH, and Top-4 MARCH. Sparse routing becomes increasingly beneficial as the context grows. At 128K tokens, Top-4 MARCH more than doubles the training throughput of dense MARCH and reduces its core runtime by roughly an order of magnitude.

It also exceeds FlashAttention-2 in throughput at this length, although vanilla Gated DeltaNet remains faster because it does not perform historical retrieval.

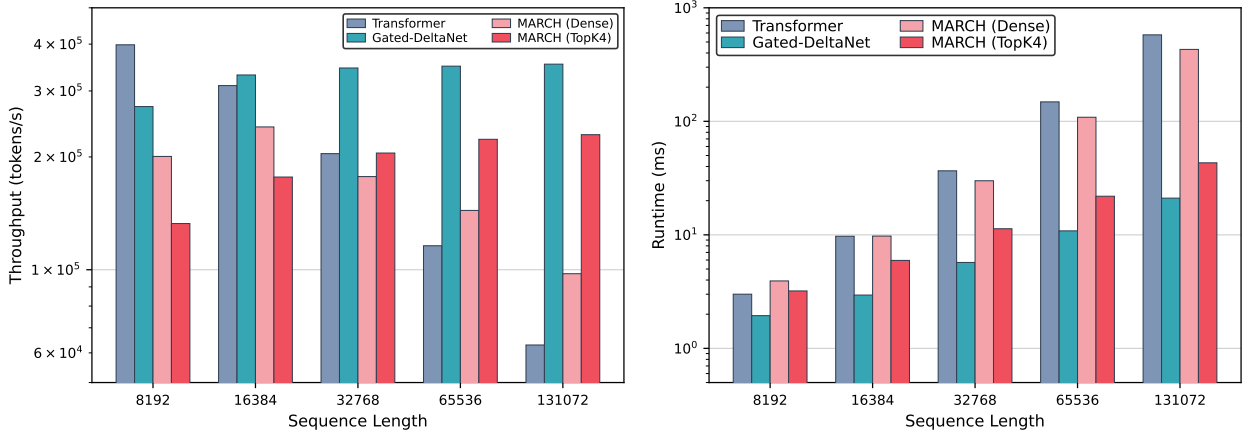


Figure 4 | Training efficiency across sequence lengths. Left: end-to-end training throughput in tokens per second (higher is better). Right: forward-backward runtime of the core sequence-mixing operation in milliseconds (lower is better). MARCH (Top-4) retains only the four highest-scoring state anchors for each token and head during historical retrieval.

Taken together, these controlled studies validate the effectiveness, generality, and scalability of MARCH’s dynamic state routing, demonstrating consistent gains in language modeling, retrieval, and length extrapolation while retaining practical efficiency at long sequence lengths.

4. Safin-1 at Scale

We next examine whether the gains observed under controlled pretraining carry over to larger foundation models, and whether Safin-1 can support persistent capability specialization through its routed state bank. We instantiate Safin-1 at the 4B and 35B-A3B scales, first evaluating the resulting models after matched continual pretraining and supervised fine-tuning, and then studying Safety from Within by adapting a persistent Safety State while keeping the shared backbone frozen.

4.1. Continual Pretraining and Supervised Fine-Tuning

4.1.1. Experimental Setup

Models and training protocol. We instantiate Safin-1 at two scales from the dense Qwen3.5-4B and Qwen3.5-35B-A3B mixture-of-experts checkpoints [37]. Both backbones employ a hybrid architecture that interleaves Gated DeltaNet (GDN) and full-attention layers. Architecturally, Safin-1 equips 16 selected GDN layers with the MARCH state-routing mechanism, comprising state anchoring and content-routed historical readout, while retaining the underlying GDN recurrence, full-attention layers, and overall hybrid layout. Historical retrieval uses Top-4 routing to control routing cost as the state bank grows.

Within each scale, the Qwen3.5 baseline and its Safin-1 counterpart are initialized from the same checkpoint and trained with matched data mixtures, token budgets, sequence lengths, and optimization settings. We first perform continual pretraining for 50B tokens on the Intern-S2 pre-training mixture [58], followed by supervised fine-tuning for a 30B-token budget using [Nemotron-Cascade-2-SFT-Data](#) [59]. The SFT mixture spans general instruction following, knowledge and reasoning,

mathematics, code, and safety alignment. Both stages use a maximum sequence length of 32K. Unless otherwise noted, we report results from the final SFT checkpoints. Complete model and training configurations are provided in Tables 11 and 12 in Appendix A.2.

Evaluation benchmarks. We evaluate the final SFT checkpoints across five evaluation categories:

- **Knowledge and reasoning:** MMLU [60], MMLU-Pro [61], and GPQA-Diamond [62].
- **Mathematics:** GSM8K [63], MATH [64], and AIME 2025.
- **Instruction following and long-context understanding:** IFEval [65] and LongBench v2 [66].
- **Code generation:** MBPP [67] and HumanEval [68].
- **Safety robustness and over-refusal:** We report attack success rate (ASR; lower is better) on WildJailbreak [69], FORTRESS [70], StrongREJECT [71], Jailbreak-R1 [72], and JailbreakBench (JBB) [73]. We additionally report over-refusal rate (ORR; lower is better) on benign prompts from XSTest [13].

These safety benchmarks evaluate Safin-1 and Qwen3.5 following the shared CPT-SFT pipeline. Section 4.2 separately studies the effect of augmenting Safin-1 with a persistent Safety State.

4.1.2. Capability and Safety Evaluation

Table 8 compares Safin-1 with its matched Qwen3.5 counterparts across capability and safety benchmarks at both model scales.

We highlight five main findings.

(i) *Safin-1 improves challenging reasoning benchmarks while preserving broad knowledge performance.* At the 4B scale, Safin-1 raises MMLU-Pro from 58.27 to 67.55 (+9.28 points) and GPQA-Diamond from 57.58 to 58.59 (+1.01 points), with only a 0.92-point decrease on MMLU. At the 35B-A3B scale, Safin-1 improves all three benchmarks, most notably raising GPQA-Diamond by 6.56 points (from 64.65 to 71.21). These results show that Safin-1 preserves broad knowledge performance while delivering larger gains on more challenging reasoning tasks.

(ii) *Safin-1 improves competition-level mathematical reasoning while maintaining performance on established mathematics benchmarks.* On AIME 2025, Safin-1 raises Avg@64 by 8.02 points at 4B and 8.22 points at 35B-A3B. Meanwhile, its performance on GSM8K and MATH remains stable: the absolute difference from the matched Qwen3.5 baseline is at most 0.68 points at 4B and 1.44 points at 35B-A3B. These results indicate that the substantial AIME gains do not come at the expense of broader mathematical capability.

(iii) *Safin-1 yields consistent gains in instruction following and long-context understanding.* Safin-1 improves both IFEval and LongBench v2 at the two model scales. At 4B, the gains are 0.19 and 0.20 points, respectively; at 35B-A3B, they increase to 1.30 and 0.40 points. Although modest, the consistent improvements across both benchmarks show that Safin-1 preserves these capabilities across both the dense 4B and larger MoE configurations.

(iv) *Safin-1 maintains or improves code generation across model scales.* At 4B, Safin-1 raises MBPP by 6.20 points while matching the Qwen3.5 baseline on HumanEval. At 35B-A3B, it improves MBPP and HumanEval by 1.00 and 1.22 points, respectively. These results show that Safin-1 preserves code-generation capability and provides additional gains across both model scales.

Across these four capability groups, the pattern remains consistent across the dense 4B and MoE 35B-A3B backbones. Gains concentrate on harder reasoning and structured-generation tasks, while established-benchmark performance remains broadly stable.

Table 8 | Capability and safety results after the matched CPT-SFT pipeline at the 4B and 35B-A3B scales. All values are percentages. Bold denotes the better result within each scale; ties are bolded for both models. ASR and ORR are lower-is-better, whereas all other metrics are higher-is-better.

Task	4B		35B-A3B	
	Qwen3.5	Safin-1	Qwen3.5	Safin-1
Knowledge and Reasoning				
MMLU	75.47	74.55	81.14	81.83
MMLU-Pro	58.27	67.55	75.33	75.98
GPQA-Diamond	57.58	58.59	64.65	71.21
Mathematics				
GSM8K	87.72	87.11	87.87	89.31
MATH	89.22	89.90	94.10	93.58
AIME 2025 (Avg@64)	55.57	63.59	71.88	80.10
Instruction Following and Long-Context Understanding				
IFEval	76.52	76.71	78.37	79.67
LongBench v2	35.19	35.39	42.54	42.94
Code				
MBPP	59.20	65.40	77.60	78.60
HumanEval	73.17	73.17	89.02	90.24
Safety Robustness				
WildJailbreak	4.40	3.60	5.20	6.80
FORTRESS	24.80	23.20	23.80	17.80
StrongREJECT	1.28	1.28	1.28	0.64
Jailbreak-R1	4.79	3.19	4.15	3.19
JBB	1.00	2.00	0.00	1.00
Average ASR	7.25	6.65	6.89	5.89
Over-Refusal (ORR ↓)				
XSTest	8.80	8.60	16.80	17.60

(v) *Safin-1 improves average safety robustness with limited change in over-refusal.* Across the five jailbreak benchmarks, average ASR falls by 0.60 points at 4B and 1.00 point at 35B-A3B, with the largest gain on FORTRESS at 35B-A3B (23.80 to 17.80). Despite small regressions on WildJailbreak and JBB at this scale, aggregate jailbreak resistance improves. XSTest ORR changes from 8.80 to 8.60 at 4B and from 16.80 to 17.60 at 35B-A3B.

Overall. Safin-1 improves seven of the ten capability benchmarks at 4B, with one tie, and nine of the ten at 35B-A3B. Together with the average reductions in jailbreak ASR, these results show that Safin-1 delivers broad capability gains and improved average safety robustness after continual pretraining and supervised fine-tuning, without a systematic trade-off in general capability or safety.

4.2. Safety from Within via a Persistent Safety State

Following Section 2.3, we instantiate a persistent Safety State across the state-bearing Safin-1 layers and evaluate whether this state-native specialization improves safety while preserving the capabilities of the frozen backbone.

4.2.1. Training Data and Experimental Setup

Training data and context construction. We construct a safety-specialization corpus from 1,000 harmful requests paired with safety-aligned responses in STAR-1 and 915 benign requests paired with helpful responses in STAR-benign [29]. Together, these sources provide 1,915 distinct target conversations. For each target, we create three context variants by prepending 0, 512, or 2,048 tokens of benign context, producing 5,745 training examples in total. Each non-empty prefix is assembled exclusively from complete benign user–assistant turns.

Beyond standard safety alignment on isolated prompts, this construction explicitly trains the Safety State under different historical-memory configurations. Because Safin-1 checkpoints the cumulative recurrent state every 512 tokens to form a state anchor, the 0-, 512-, and 2,048-token conditions present the same target with no context-derived state anchor, one state anchor, and a bank of multiple state anchors, respectively. Reusing the same target supervision across these conditions controls for target content while varying the amount and configuration of historical memory. This encourages the learned Safety State to remain effective across different context lengths and state-anchor bank compositions. Finally, joint supervision on harmful and benign targets couples safety with helpfulness: the model must produce safety-aligned responses to harmful requests while continuing to answer legitimate requests, thereby reducing the tendency toward indiscriminate refusal.

Persistent Safety State training. We initialize the 4B and 35B-A3B experiments from the corresponding Safin-1 SFT checkpoints and freeze all language-model parameters. The only trainable component is a persistent Safety State, represented by a persistent state inserted into the routable memory-state bank of each state-bearing Safin-1 layer. We optimize the persistent states with standard token-level causal cross-entropy.

Training-matched LoRA control. For a controlled comparison, we train a rank-8 LoRA control [38] from the same SFT checkpoint using the same data, optimizer, batch size, and number of updates. The Safety State lane is disabled for this control, while periodic state anchoring and content-routed historical retrieval remain active. We use $r = 8$, $\alpha = 32$, and dropout 0.05 without bias training or RSLORA. At 4B, adapters cover the recurrent, full-attention, and MLP projections; at 35B-A3B, they cover the recurrent and full-attention projections but no dense, expert, or shared-expert MLP projections.

Evaluation protocol. We reuse the safety suite introduced in Section 4.1. The five jailbreak benchmarks are evaluated by attack success rate (ASR), while XSTest measures the over-refusal rate (ORR) on benign prompts; lower is better for both. To distinguish genuine safety improvement from indiscriminate refusal or broader capability loss, we additionally report retention on MMLU-Pro, MATH (Minerva), AIME 2025 (Avg@64), and MBPP. Optimizer settings and numerical precision are summarized in Table 13 in Appendix A.3.

4.2.2. Safety–Utility Trade-off

We compare the persistent Safety State with the unadapted Safin-1 checkpoint and the training-matched LoRA control at both model scales. The comparison covers jailbreak robustness, over-refusal on benign prompts, and retention of general capabilities. Table 9 summarizes the results, from which three findings emerge.

Table 9 | Safety–utility trade-off of the persistent Safety State and training-matched rank-8 LoRA control for Safin-1 at the 4B and 35B-A3B scales.

Task	Safin-1 (4B)			Safin-1 (35B-A3B)		
	Base	Safety State	LoRA	Base	Safety State	LoRA
Safety Robustness (ASR ↓)						
WildJailbreak	3.60	2.40	1.60	6.80	1.60	1.20
FORTRESS	23.20	13.60	20.20	17.80	11.80	12.40
StrongREJECT	1.28	1.60	2.88	0.64	0.32	0.64
JailBreak-R1	3.19	1.60	0.96	3.19	0.32	0.96
JBB	2.00	0.00	1.00	1.00	0.00	0.00
Average ASR	6.65	3.84	5.33	5.89	2.81	3.04
Over-refusal (ORR ↓)						
XSTest	8.60	9.00	13.60	17.60	19.60	27.60
Capability Retention (↑)						
MMLU-Pro	67.55	66.49	65.55	75.98	72.61	74.92
MATH	89.90	87.54	87.42	93.58	93.32	93.12
AIME 2025 (Avg@64)	63.59	60.99	60.42	80.10	74.21	72.61
MBPP	65.40	64.40	62.80	78.60	74.40	76.80
Average	71.61	69.86	69.05	82.07	78.64	79.36

Note. Base is the unadapted Safin-1 checkpoint. Averages are unweighted means across the five safety or four capability benchmarks, respectively. Bold compares Safety State with LoRA within each scale; ties are bolded for both. All values are percentages.

(i) *The persistent Safety State achieves the lowest average jailbreak ASR at both model scales.* At 4B, it reduces Average ASR from 6.65 to 3.84, a 2.81-point improvement over the unadapted checkpoint, whereas LoRA reaches 5.33. The Safety State improves four of the five jailbreak benchmarks, with only a slight 0.32-point regression on StrongREJECT. At 35B-A3B, it improves all five benchmarks and lowers Average ASR from 5.89 to 2.81, compared with 3.04 for LoRA. The largest per-benchmark gain occurs on FORTRESS at both scales, with ASR reductions of 9.60 and 6.00 points at 4B and 35B-A3B, respectively.

(ii) *The Safety State achieves a better safety–over-refusal trade-off than the training-matched LoRA control.* On XSTest, the Safety State increases ORR by 0.40 points at 4B (from 8.60 to 9.00) and by 2.00 points at 35B-A3B (from 17.60 to 19.60). Under matched training conditions, LoRA produces substantially larger increases of 5.00 and 10.00 points, reaching 13.60 and 27.60, respectively. At the same time, the Safety State achieves lower Average ASR than LoRA by 1.49 points at 4B and 0.23 points at 35B-A3B. These results show that the Safety State improves jailbreak robustness while introducing substantially less over-refusal than the LoRA control.

(iii) *The Safety State preserves capabilities more consistently than LoRA at 4B, while retention at 35B-A3B is task-dependent.* At 4B, the Safety State remains within 2.60 points of the unadapted checkpoint across all four capability benchmarks and outperforms LoRA on each of them. At 35B-A3B, the results are mixed. The Safety State nearly matches the unadapted checkpoint on MATH (93.32 versus 93.58) and retains more capability than LoRA on MATH and AIME 2025, whereas LoRA performs better on MMLU-Pro and MBPP. Thus, the Safety State provides a clear capability-retention advantage over LoRA at 4B, while the larger model reveals no uniform winner across capability tasks. Improving knowledge and code retention at the 35B-A3B scale remains an important direction for future work. Overall, Safety State provides a stronger safety–utility trade-off than matched LoRA.

5. Related Works

Efficient long-context sequence modeling. Efficient attention methods reduce the cost of full self-attention through restricted receptive fields, kernel approximations, systems-level optimization, or sparse selection. Local and sliding-window attention bound the neighborhood available to each query [74, 75], while Performer, Nyströmformer, and Linear Attention replace or approximate the softmax kernel with feature maps that permit associative computation [76, 77, 15]. FlashAttention-2 accelerates exact dense attention through hardware-aware execution, while sequence-parallel algorithms improve the scaling of linear attention [78, 79]. During autoregressive decoding, however, exact dense attention still retains a token-level key-value cache that grows with the context.

Sparse attention preserves content-based retrieval while reducing the number of token-level interactions. Early methods impose predefined connectivity: Sparse Transformer factorizes the attention graph, whereas Longformer and BigBird combine local windows with global or random connections [80, 81, 82]. Later systems make selection input dependent. Routing Transformer clusters tokens by content, H₂O evicts low-utility cache entries, and Quest selects KV-cache pages according to the current query [83, 84, 85]. Recent trainable designs make routing part of the model itself: MoBA assigns queries to relevant key-value blocks, while Native Sparse Attention combines local, compressed, and selectively retrieved branches with hardware-aligned kernels [39, 40]. These methods differ in their selection rules and retrieval granularity, but continue to operate over token- or block-level key-value representations. Safin-1 instead retrieves compact snapshots of the model’s recurrent-state trajectory, placing long-range routing over state-level memories.

Recurrent state and expandable memory. State-space models and gated linear recurrences compress the causal prefix into a recurrent state, replacing an expanding token cache with fixed-size updates and reads. Linear Attention exposes this connection through outer-product state updates and query-based retrieval [15, 86, 16]. Structured and selective state-space models improve the underlying transition dynamics, while RetNet, RWKV, HGRN2, and related recurrent formulations combine associative memory with structured recurrence [87, 17, 16, 88, 89, 90, 91, 92]. Gated Linear Attention introduces input-dependent decay, and DeltaNet-style models use delta-rule corrections to refine associative updates [93, 19, 34, 94, 95]. Despite these advances, most recurrent models expose a single fixed-capacity state, creating a retrieval bottleneck when earlier associations are attenuated or overwritten [20, 21, 96].

Expandable-memory methods address this bottleneck by increasing the number or organization of accessible memory units. Multi-State RNNs, HGRN2, and Log-Linear Attention maintain multiple or hierarchically organized recurrent states [97, 90, 25]. Other systems decouple memory capacity from dense computation through routed memory experts, sparsely accessed state banks, or product-key memories [24, 22, 98, 99, 100, 23, 101]. Context-compression methods use a different retrieval unit: learned summary or gist tokens compress earlier spans, and selective schemes reopen relevant compressed chunks when needed [102, 103, 104, 105, 106, 107]. These approaches therefore span recurrent-state expansion, parameterized memory, and token-level context compression. MARCH [28], the architecture underlying Safin-1, takes a complementary approach: it preserves cumulative snapshots of the model’s native recurrent state, expands the addressable state bank with context length, and retrieves historical states through content-conditioned routing. The resulting memory units remain native to the recurrent computation while becoming individually addressable by later tokens. Unlike approaches that replace the recurrent state with a larger learned memory, this design preserves the original update path while adding addressability over its trajectory. State evolution and historical access are therefore separated: the recurrence determines how the current state is updated, while routing determines which earlier internal representations become available to later tokens.

Data-efficient and modular safety alignment. Safety alignment can be data efficient: Safety-Tuned LLaMAs show substantial gains from a few hundred demonstrations, while STAR-1 aligns reasoning models with 1K curated examples [1, 29]. Robustness must nevertheless be evaluated alongside over-refusal, as emphasized by adversarial suites such as WildJailbreak and benign-prompt diagnostics such as XSTest [69, 13]. Parameter-efficient methods preserve or restore safety through low-rank adaptation, pruning, or post-hoc subspace composition [108, 2, 109, 110, 111]. Together, these studies show that reducing attack success is insufficient if alignment induces blanket refusal or erodes general capability. They motivate our mixed harmful and benign supervision, as well as the joint reporting of jailbreak success, over-refusal, and capability retention.

A complementary line of work changes the locus of control from backbone weights to model representations. ReFT learns hidden-state interventions, while Representation Engineering, Contrastive Activation Addition, and refusal-direction analyses manipulate behavior through low-dimensional activation directions [112, 113, 114, 115]. Safety-focused approaches transfer, sparsely adjust, or input-condition these directions [4, 116, 117, 118, 119, 120]. Related modular work explores other control interfaces: Circuit Breakers reroute internal representations, MoGU routes among model variants, SAFEx studies safety-critical experts, and Single Token Alignment uses a learned input prefix [121, 122, 123, 3]. Safin-1 differs primarily in the locus of specialization. Its Safety State is learned from external harmful and benign supervision, but is represented as a persistent matrix-valued recurrent state and invoked through the model’s native routing pathway alongside context-derived memories. Unlike low-rank adaptation, it does not modify the shared backbone; unlike per-token activation steering, it persists as a routable memory candidate. The backbone remains frozen during specialization; once learned, the Safety State can be attached or removed, and the router assigns it a token- and context-dependent contribution. This provides a state-native interface for test-time adaptation.

6. Discussion and Conclusion

Model memory as an architectural substrate. Safin-1 demonstrates that recurrent state can serve not only as a transient summary of the prefix, but also as an addressable substrate for long-horizon memory. Its underlying MARCH state-routing mechanism preserves selected points along the recurrent trajectory and makes them individually addressable. The consistent gains across GDN, KDA, and GDN2 indicate that historical-state access is complementary to the recurrent update rule, while the 4B and 35B-A3B Safin-1 experiments show that these benefits persist after continual pretraining and supervised fine-tuning. Recurrent memory quality therefore depends not only on how information is written into the current state, but also on whether useful earlier versions remain accessible. Because state anchors are compressed cumulative memories rather than token-level records, this design complements rather than replicates the exact recall path of full attention.

State-native specialization for Safety from Within. In Safin-1, the same routed state bank can host both context-derived anchors and learned persistent capability states. Safety State provides an initial demonstration: with the backbone frozen, it is retrieved through the native router and achieves a better aggregate safety-over-refusal trade-off than training-matched LoRA. Here, *Safety from Within* refers to where safety is represented and invoked, not to where its supervision originates: the state is still learned from external harmful and benign examples. Its ability to be attached or removed enables modular specialization over a shared backbone, but does not make safety immutable or universally robust. Detachability also creates an integrity requirement: both the persistent state and its routing path must be protected against removal, substitution, or adversarial suppression. The



same interface may extend to capabilities beyond safety, although whether multiple persistent states can coexist and compose reliably remains an open question.

Limitations and research agenda. Architecturally, the state bank grows with context length, and sparse routing reduces reading cost but not state storage; fixed-interval anchoring may also retain redundant checkpoints. Empirically, our large-scale evidence is limited to two Qwen3.5 configurations under a single CPT-SFT pipeline. The safety study uses a small corpus and a limited set of English benchmarks, does not cover adaptive or multilingual attacks or state tampering, and shows task-dependent capability retention at 35B-A3B. Future work should develop adaptive anchor creation, consolidation, and eviction; evaluate broader model families and context regimes; and study how multiple persistent states can be composed without routing conflicts or capability interference. Beyond improving the current state-routing design, it is essential to investigate alternative and complementary architectures that make safety representable, maintainable, and selectively invocable through a model’s native computation.

Conclusion. We introduced Safin-1, a family of foundation models that treats model state as both addressable context memory and an interface for adaptable capabilities. Controlled 0.8B experiments validate the underlying state-routing mechanism across recurrent update rules, while matched 4B and 35B-A3B CPT-SFT experiments show that its gains persist at larger scale. With the backbone frozen, Safety State reduces the unweighted average ASR across five jailbreak benchmarks by 42.3% and 52.3% at 4B and 35B-A3B, respectively, while yielding lower XSTest ORR than the training-matched rank-8 LoRA controls. These results provide initial architecture-level evidence for *Safety from Within*, rather than establishing a definitive solution. Exploring alternative and complementary architectural routes toward this broader objective remains essential.

Project Contributors

This work was led by the Shanghai Artificial Intelligence Laboratory, with contributions from the authors listed below in contribution order.

Ming Zhang*	Kaisen Yang*	Shu Yu	Ermo Hua
Zhekai Chen	Cheng Jin	Jingnan Zheng	Yi Zhang
Dongcheng Zhang	Zhongtian Ma	Jiawei Zhou	Sirui Chen
Qiaosheng Zhang	Xiang Wang	Ning Ding	Xia Hu
Bowen Zhou	Youbang Sun [‡]	Chaochao Lu [†]	

* Equal Contribution. [‡] Technical Lead. [†] Project Lead.

References

- [1] Federico Bianchi et al. “Safety-Tuned LLaMAs: Lessons From Improving the Safety of Large Language Models that Follow Instructions”. In: *International Conference on Learning Representations*. 2024. URL: https://proceedings.iclr.cc/paper_files/paper/2024/hash/9178ece87f0a5d6558f49f43ec1e8a1a-Abstract-Conference.html.
- [2] Mingjie Li et al. “SaLoRA: Safety-Alignment Preserved Low-Rank Adaptation”. In: *International Conference on Learning Representations*. 2025. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/e24d9d028e3c7f6f13e6032919ee021e-Abstract-Conference.html.
- [3] Wenqian Yu et al. “You Only Need One Single Token to Refine Safety Alignment”. In: *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 2026, pp. 13529–13545. DOI: [10.18653/v1/2026.findings-acl.662](https://doi.org/10.18653/v1/2026.findings-acl.662). URL: <https://aclanthology.org/2026.findings-acl.662/>.
- [4] Pengyu Wang et al. “InferAligner: Inference-Time Alignment for Harmlessness through Cross-Model Guidance”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 10460–10479. DOI: [10.18653/v1/2024.emnlp-main.585](https://doi.org/10.18653/v1/2024.emnlp-main.585). URL: <https://aclanthology.org/2024.emnlp-main.585/>.
- [5] Youbang Sun et al. “Position: Safe AI Should be Resistant and Resilient in an Evolving World”. In: *Forty-third International Conference on Machine Learning Position Paper Track*. 2026. arXiv: [2509.06786](https://arxiv.org/abs/2509.06786) [cs.LG]. URL: <https://openreview.net/forum?id=ApUb0ToKMN>.
- [6] Chao Yang et al. “Towards AI-45° Law: A Roadmap to Trustworthy AGI”. In: *arXiv preprint arXiv:2412.14186* (2024). DOI: [10.48550/arXiv.2412.14186](https://doi.org/10.48550/arXiv.2412.14186). URL: <https://arxiv.org/abs/2412.14186>.
- [7] Shanghai AI Lab et al. “SafeWork-R1: Coevolving Safety and Intelligence under the AI-45 Law”. In: *arXiv preprint arXiv:2507.18576* (2025).
- [8] Xiaoyu Wen et al. “JailbreakSkill: Scaling Automated Red-Teaming with Reusable and Ever-Evolving Skills”. In: *arXiv preprint arXiv:2608.16465* (2026).
- [9] Weiwei Qi et al. “DARWIN: Evolving Jailbreak Adversary and Guardrail for LLM Safety Evaluation and Protection”. In: *arXiv preprint arXiv:2607.19829* (2026).
- [10] Yu Li et al. “EvoDefense: Co-Evolving Black-Box Defense with Large Language Models”. In: *arXiv preprint arXiv:2605.31140* (2026).
- [11] Churui Zeng et al. “TRACE: Task-Aware Adaptive Self-Evolving Agentic Jailbreaking”. In: *arXiv preprint arXiv:2605.30883* (2026).
- [12] Xiaoyu Wen et al. “MAGIC: A co-evolving attacker-defender adversarial game for robust LLM safety”. In: *arXiv preprint arXiv:2602.01539* (2026).
- [13] Paul Röttger et al. “XSTEST: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models”. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 2024, pp. 5377–5400. DOI: [10.18653/v1/2024.naacl-long.301](https://doi.org/10.18653/v1/2024.naacl-long.301). URL: <https://aclanthology.org/2024.naacl-long.301/>.

- [14] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. Ed. by Isabelle Guyon et al. 2017, pp. 5998–6008. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- [15] Angelos Katharopoulos et al. “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention”. In: *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 5156–5165. URL: <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- [16] Tri Dao and Albert Gu. “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 10041–10071. URL: <https://proceedings.mlr.press/v235/dao24a.html>.
- [17] Albert Gu and Tri Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. In: *arXiv preprint arXiv:2312.00752* (2023). URL: <https://arxiv.org/abs/2312.00752>.
- [18] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. *Linear Transformers Are Secretly Fast Weight Programmers*. 2021. arXiv: 2102.11174 [cs.LG]. URL: <https://arxiv.org/abs/2102.11174>.
- [19] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. “Gated Delta Networks: Improving Mamba2 with Delta Rule”. In: *International Conference on Learning Representations*. 2025. URL: <https://openreview.net/forum?id=r8H7xhYPwz>.
- [20] Simran Arora et al. “Zoology: Measuring and improving recall in efficient language models”. In: *Proceedings of 12th International Conference on Learning Representations (ICLR)*. ICLR. 2024. URL: <https://openreview.net/forum?id=LY3ukUANKo>.
- [21] Simran Arora et al. “Simple Linear Attention Language Models Balance the Recall-Throughput Tradeoff”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 1763–1840. URL: <https://proceedings.mlr.press/v235/arora24a.html>.
- [22] Yuqi Pan et al. “Scaling Linear Attention with Sparse State Expansion”. In: *arXiv preprint arXiv:2507.16577* (2025). URL: <https://arxiv.org/abs/2507.16577>.
- [23] Loïc Cabannes et al. “Sparse Delta Memory: Scaling the State of Linear RNNs through Sparsity”. In: *arXiv preprint arXiv:2607.07386* (2026). URL: <https://arxiv.org/abs/2607.07386>.
- [24] Jusen Du et al. “MoM: Linear Sequence Modeling with Mixture-of-Memories”. In: *arXiv preprint arXiv:2502.13685* (2025). URL: <https://arxiv.org/abs/2502.13685>.
- [25] Han Guo et al. “Log-Linear Attention”. In: *International Conference on Learning Representations*. 2026. URL: <https://openreview.net/forum?id=m0JgZWkXKW>.
- [26] Xin Wang et al. “Dynamic Linear Attention”. In: *arXiv preprint arXiv:2606.10650* (2026). URL: <https://arxiv.org/abs/2606.10650>.
- [27] Ali Behrouz et al. “Memory Caching: RNNs with Growing Memory”. In: *Forty-third International Conference on Machine Learning*. 2026. URL: <https://arxiv.org/abs/2602.24281>.

- [28] Ming Zhang et al. “MARCH: Scaling Recurrent Memory with Content-Routed State Anchors”. In: *arXiv preprint arXiv:2608.12435* (2026). DOI: [10.48550/arXiv.2608.12435](https://arxiv.org/abs/2608.12435). URL: <https://arxiv.org/abs/2608.12435>.
- [29] Zijun Wang et al. “STAR-1: Safer Alignment of Reasoning LLMs with 1K Data”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 44. 2026, pp. 37988–37997. DOI: [10.1609/aaai.v40i44.41136](https://ojs.aaai.org/index.php/AAAI/article/view/41136). URL: <https://ojs.aaai.org/index.php/AAAI/article/view/41136>.
- [30] David Dalrymple et al. “Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems”. In: *arXiv preprint arXiv:2405.06624* (2024).
- [31] Damiano Fornasiero et al. *The Scientist AI: Safe by Design, by Not Desiring*. Preprint. Damiano Fornasiero and Oliver Richardson contributed equally. LawZero, 2026. URL: <https://lawzero.org/en/publication/scientist-ai-safe-design-not-desiring>.
- [32] Xin Tan et al. “Towards Safe and Trustworthy Embodied AI: Foundations, Status, and Prospects”. In: *OpenReview* (2025). URL: <https://openreview.net/pdf?id=Eu6Yt21Alv>.
- [33] Kimi Team. “Kimi Linear: An Expressive, Efficient Attention Architecture”. In: *arXiv preprint arXiv:2510.26692* (2025). URL: <https://arxiv.org/abs/2510.26692>.
- [34] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. *Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention*. 2026. arXiv: [2605.22791](https://arxiv.org/abs/2605.22791) [cs.LG]. URL: <https://arxiv.org/abs/2605.22791>.
- [35] Yushi Bai et al. “Longbench: A bilingual, multitask benchmark for long context understanding”. In: *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*. 2024, pp. 3119–3137. DOI: [10.18653/v1/2024.acl-long.172](https://aclanthology.org/2024.acl-long.172/). URL: <https://aclanthology.org/2024.acl-long.172/>.
- [36] Cheng-Ping Hsieh et al. “RULER: What’s the Real Context Size of Your Long-Context Language Models?” In: *First Conference on Language Modeling*. 2024. URL: <https://openreview.net/forum?id=kIoBbc76Sy>.
- [37] Qwen Team. *Qwen3.5: Towards Native Multimodal Agents*. 2026. URL: <https://qwen.ai/blog?id=qwen3.5>.
- [38] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations*. 2022. URL: <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [39] Enzhe Lu et al. “MoBA: Mixture of Block Attention for Long-Context LLMs”. In: *arXiv preprint arXiv:2502.13189* (2025). URL: <https://arxiv.org/abs/2502.13189>.
- [40] Jingyang Yuan et al. “Native sparse attention: Hardware-aligned and natively trainable sparse attention”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025, pp. 23078–23097. DOI: [10.18653/v1/2025.acl-long.1126](https://aclanthology.org/2025.acl-long.1126/). URL: <https://aclanthology.org/2025.acl-long.1126/>.
- [41] Xiang Hu et al. “Hierarchical Sparse Attention Done Right: Toward Infinite Context Modeling”. In: *arXiv preprint arXiv:2607.02980* (2026). URL: <https://arxiv.org/abs/2607.02980>.
- [42] Tri Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 16344–16359. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

- [43] Denis Paperno et al. “The LAMBADA dataset: Word prediction requiring a broad discourse context”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Katrin Erk and Noah A. Smith. Berlin, Germany: Association for Computational Linguistics, 2016, pp. 1525–1534. DOI: [10.18653/v1/P16-1144](https://doi.org/10.18653/v1/P16-1144). URL: <https://aclanthology.org/P16-1144>.
- [44] Yonatan Bisk et al. “PIQA: Reasoning about Physical Commonsense in Natural Language”. In: *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 2020, pp. 7432–7439. DOI: [10.1609/aaai.v34i05.6239](https://doi.org/10.1609/aaai.v34i05.6239). URL: <https://ojs.aaai.org/index.php/AAAI/article/view/6239>.
- [45] Rowan Zellers et al. “HellaSwag: Can a Machine Really Finish Your Sentence?”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Ed. by Anna Korhonen, David Traum, and Lluís Màrquez. Florence, Italy: Association for Computational Linguistics, 2019, pp. 4791–4800. DOI: [10.18653/v1/P19-1472](https://doi.org/10.18653/v1/P19-1472). URL: <https://aclanthology.org/P19-1472>.
- [46] Keisuke Sakaguchi et al. “WinoGrande: An Adversarial Winograd Schema Challenge at Scale”. In: *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 2020, pp. 8732–8740. DOI: [10.1609/aaai.v34i05.6399](https://doi.org/10.1609/aaai.v34i05.6399). URL: <https://ojs.aaai.org/index.php/AAAI/article/view/6399>.
- [47] Peter Clark et al. “Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge”. In: *ArXiv preprint abs/1803.05457* (2018). URL: <https://arxiv.org/abs/1803.05457>.
- [48] Todor Mihaylov et al. “Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2018, pp. 2381–2391. DOI: [10.18653/v1/D18-1260](https://doi.org/10.18653/v1/D18-1260). URL: <https://aclanthology.org/D18-1260/>.
- [49] Alon Talmor et al. “CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pp. 4149–4158. DOI: [10.18653/v1/N19-1421](https://doi.org/10.18653/v1/N19-1421). URL: <https://aclanthology.org/N19-1421/>.
- [50] Pranav Rajpurkar, Robin Jia, and Percy Liang. “Know What You Don’t Know: Unanswerable Questions for SQuAD”. In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Ed. by Iryna Gurevych and Yusuke Miyao. Melbourne, Australia: Association for Computational Linguistics, 2018, pp. 784–789. DOI: [10.18653/v1/P18-2124](https://doi.org/10.18653/v1/P18-2124). URL: <https://aclanthology.org/P18-2124>.
- [51] Mandar Joshi et al. “TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension”. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Regina Barzilay and Min-Yen Kan. Vancouver, Canada: Association for Computational Linguistics, 2017, pp. 1601–1611. DOI: [10.18653/v1/P17-1147](https://doi.org/10.18653/v1/P17-1147). URL: <https://aclanthology.org/P17-1147>.

- [52] Colin Lockard, Prashant Shiralkar, and Xin Luna Dong. “OpenCeres: When Open Information Extraction Meets the Semi-Structured Web”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 3047–3056. DOI: [10.18653/v1/N19-1309](https://doi.org/10.18653/v1/N19-1309). URL: <https://aclanthology.org/N19-1309>.
- [53] Simran Arora et al. *Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes*. 2023. URL: <https://arxiv.org/abs/2304.09433>.
- [54] Tom Kwiatkowski et al. “Natural Questions: A Benchmark for Question Answering Research”. In: *Transactions of the Association for Computational Linguistics* 7 (2019). Ed. by Lillian Lee et al., pp. 452–466. DOI: [10.1162/tac1_a_00276](https://doi.org/10.1162/tac1_a_00276). URL: <https://aclanthology.org/Q19-1026/>.
- [55] Dheeru Dua et al. “DROP: A Reading Comprehension Benchmark Requiring Discrete Reasoning Over Paragraphs”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 2368–2378. DOI: [10.18653/v1/N19-1246](https://doi.org/10.18653/v1/N19-1246). URL: <https://aclanthology.org/N19-1246>.
- [56] Leo Gao et al. *A framework for few-shot language model evaluation*. Version v0.0.1. 2021. DOI: [10.5281/zenodo.5371629](https://doi.org/10.5281/zenodo.5371629). URL: <https://zenodo.org/records/5371629>.
- [57] Peter M. Fenwick. “A New Data Structure for Cumulative Frequency Tables”. In: *Software: Practice and Experience* 24.3 (1994), pp. 327–336. DOI: [10.1002/spe.4380240306](https://doi.org/10.1002/spe.4380240306). URL: <https://doi.org/10.1002/spe.4380240306>.
- [58] Lei Bai, Jiaqi Cao, Chiyu Chen, et al. “Intern-S2-Preview: Scientific Agentic Foundation Model”. In: *arXiv preprint arXiv:2608.13505* (2026). URL: <https://arxiv.org/abs/2608.13505>.
- [59] Zhuolin Yang et al. “Nemotron-Cascade 2: Post-Training LLMs with Cascade RL and Multi-Domain On-Policy Distillation”. In: *arXiv preprint arXiv:2603.19220* (2026). URL: <https://arxiv.org/abs/2603.19220>.
- [60] Dan Hendrycks et al. “Measuring Massive Multitask Language Understanding”. In: *International Conference on Learning Representations*. 2021. URL: <https://arxiv.org/abs/2009.03300>.
- [61] Yubo Wang et al. “MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark”. In: *arXiv preprint arXiv:2406.01574* (2024). URL: <https://arxiv.org/abs/2406.01574>.
- [62] David Rein et al. “GPQA: A Graduate-Level Google-Proof Q&A Benchmark”. In: *arXiv preprint arXiv:2311.12022* (2024). URL: <https://arxiv.org/abs/2311.12022>.
- [63] Karl Cobbe et al. “Training Verifiers to Solve Math Word Problems”. In: *arXiv preprint arXiv:2110.14168* (2021). URL: <https://arxiv.org/abs/2110.14168>.
- [64] Dan Hendrycks et al. “Measuring Mathematical Problem Solving With the MATH Dataset”. In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021. URL: <https://arxiv.org/abs/2103.03874>.
- [65] Jeffrey Zhou et al. “Instruction-Following Evaluation for Large Language Models”. In: *arXiv preprint arXiv:2311.07911* (2023). URL: <https://arxiv.org/abs/2311.07911>.

- [66] Yushi Bai et al. “Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025, pp. 3639–3664. DOI: [10.18653/v1/2025.acl-long.183](https://doi.org/10.18653/v1/2025.acl-long.183). URL: <https://aclanthology.org/2025.acl-long.183/>.
- [67] Jacob Austin et al. “Program Synthesis with Large Language Models”. In: *arXiv preprint arXiv:2108.07732* (2021). URL: <https://arxiv.org/abs/2108.07732>.
- [68] Mark Chen et al. “Evaluating Large Language Models Trained on Code”. In: *arXiv preprint arXiv:2107.03374* (2021). URL: <https://arxiv.org/abs/2107.03374>.
- [69] Liwei Jiang et al. “WildTeaming at Scale: From In-the-Wild Jailbreaks to (Adversarially) Safer Language Models”. In: *arXiv preprint arXiv:2406.18510* (2024). URL: <https://arxiv.org/abs/2406.18510>.
- [70] Christina Q. Knight et al. “FORTRESS: Frontier Risk Evaluation for National Security and Public Safety”. In: *arXiv preprint arXiv:2506.14922* (2025). URL: <https://arxiv.org/abs/2506.14922>.
- [71] Alexandra Souly et al. “A StrongREJECT for Empty Jailbreaks”. In: *arXiv preprint arXiv:2402.10260* (2024). URL: <https://arxiv.org/abs/2402.10260>.
- [72] Weiyang Guo et al. “Jailbreak-R1: Exploring the Jailbreak Capabilities of LLMs via Reinforcement Learning”. In: *arXiv preprint arXiv:2506.00782* (2025). URL: <https://arxiv.org/abs/2506.00782>.
- [73] Patrick Chao et al. “JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024. URL: <https://arxiv.org/abs/2404.01318>.
- [74] Bailin Wang et al. *RATTENTION: Towards the Minimal Sliding Window Size in Local-Global Attention Models*. 2025. arXiv: [2506.15545](https://arxiv.org/abs/2506.15545) [cs.CL]. URL: <https://arxiv.org/abs/2506.15545>.
- [75] Loïc Cabannes et al. *Short window attention enables long-term memorization*. 2025. arXiv: [2509.24552](https://arxiv.org/abs/2509.24552) [cs.LG]. URL: <https://arxiv.org/abs/2509.24552>.
- [76] Krzysztof Marcin Choromanski et al. “Rethinking Attention with Performers”. In: *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL: <https://openreview.net/forum?id=Ua6zuk0WRH>.
- [77] Yunyang Xiong et al. “Nyströmformer: A Nyström-Based Algorithm for Approximating Self-Attention”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 35.16 (2021), pp. 14138–14148. DOI: [10.1609/aaai.v35i16.17664](https://doi.org/10.1609/aaai.v35i16.17664). URL: <https://ojs.aaai.org/index.php/AAAI/article/view/17664>.
- [78] Tri Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”. In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- [79] Weigao Sun et al. “Linear Attention Sequence Parallelism”. In: *ArXiv preprint abs/2404.02882* (2024). URL: <https://arxiv.org/abs/2404.02882>.
- [80] Rewon Child et al. “Generating Long Sequences with Sparse Transformers”. In: *arXiv preprint arXiv:1904.10509* (2019). URL: <https://arxiv.org/abs/1904.10509>.
- [81] Iz Beltagy, Matthew E Peters, and Arman Cohan. “Longformer: The long-document transformer”. In: *arXiv preprint arXiv:2004.05150* (2020). URL: <https://arxiv.org/abs/2004.05150>.

- [82] Manzil Zaheer et al. “Big Bird: Transformers for Longer Sequences”. In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. Ed. by Hugo Larochelle et al. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/c8512d142a2d849725f31a9a7a361ab9-Abstract.html>.
- [83] Aurko Roy et al. “Efficient Content-Based Sparse Attention with Routing Transformers”. In: *Transactions of the Association for Computational Linguistics 9* (2021). Ed. by Brian Roark and Ani Nenkova, pp. 53–68. DOI: 10.1162/tac1_a_00353. URL: <https://aclanthology.org/2021.tac1-1.4>.
- [84] Zhenyu Zhang et al. “H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models”. In: *Advances in Neural Information Processing Systems 36* (2023), pp. 34661–34710. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/6ceefa7b15572587b78ecfceb2827f8-Abstract-Conference.html.
- [85] Jiaming Tang et al. “Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 47901–47911. URL: <https://proceedings.mlr.press/v235/tang241.html>.
- [86] Yuhong Chou et al. “MetaLA: Unified Optimal Linear Approximation to Softmax Attention Map”. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. 2024. URL: <https://openreview.net/forum?id=Y8YVCOMEpz>.
- [87] Albert Gu, Karan Goel, and Christopher Ré. “Efficiently Modeling Long Sequences with Structured State Spaces”. In: *The Tenth International Conference on Learning Representations*. 2022. URL: <https://openreview.net/forum?id=uYLFoz1vlAC>.
- [88] Yutao Sun et al. “Retentive network: A successor to transformer for large language models”. In: *ArXiv preprint abs/2307.08621* (2023). URL: <https://arxiv.org/abs/2307.08621>.
- [89] Bo Peng et al. “RWKV: Reinventing RNNs for the Transformer Era”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023, pp. 14048–14077. DOI: 10.18653/v1/2023.findings-emnlp.936. URL: <https://aclanthology.org/2023.findings-emnlp.936/>.
- [90] Zhen Qin et al. “HGRN2: Gated Linear RNNs with State Expansion”. In: *First Conference on Language Modeling* (2024). URL: <https://openreview.net/forum?id=y6SqbJfCSk>.
- [91] Antonio Orvieto et al. “Resurrecting recurrent neural networks for long sequences”. In: *International Conference on Machine Learning*. PMLR. 2023, pp. 26670–26698. URL: <https://proceedings.mlr.press/v202/orvieto23a.html>.
- [92] Bo Liu et al. “Longhorn: State Space Models are Amortized Online Learners”. In: *arXiv preprint arXiv:2407.14207* (2024). URL: <https://arxiv.org/abs/2407.14207>.
- [93] Songlin Yang et al. “Parallelizing Linear Transformers with the Delta Rule over Sequence Length”. In: *Advances in Neural Information Processing Systems* (2024). URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/d13a3eae72366e61dfdc7eea82eeb685-Abstract-Conference.html.
- [94] Julien Siems et al. *DeltaProduct: Improving State-Tracking in Linear RNNs via Householder Products*. *arXiv preprint arXiv:2502.10297*, 2025. 2025. URL: <https://arxiv.org/abs/2502.10297>.

- [95] Riccardo Grazzi et al. “Unlocking State-Tracking in Linear RNNs Through Negative Eigenvalues”. In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/5a0ce3abb720b740419e193c87afd080-Abstract-Conference.html.
- [96] Kaiyue Wen, Xingyu Dang, and Kaifeng Lyu. “RNNs are not Transformers (Yet): The Key Bottleneck on In-context Retrieval”. In: *ArXiv preprint abs/2402.18510* (2024). URL: <https://arxiv.org/abs/2402.18510>.
- [97] Matanel Oren et al. “Transformers are Multi-State RNNs”. In: *ArXiv preprint abs/2401.06104* (2024). URL: <https://arxiv.org/abs/2401.06104>.
- [98] Guillaume Lample et al. *Large Memory Layers with Product Keys*. 2019. arXiv: 1907.05242 [cs.CL]. URL: <https://arxiv.org/abs/1907.05242>.
- [99] Vincent-Pierre Berges et al. *Memory Layers at Scale*. 2024. arXiv: 2412.09764 [cs.CL]. URL: <https://arxiv.org/abs/2412.09764>.
- [100] Tianyu Zhao and Llion Jones. *Fast-weight Product Key Memory*. 2026. arXiv: 2601.00671 [cs.CL]. URL: <https://arxiv.org/abs/2601.00671>.
- [101] Arshia Afzal et al. “Raven: High-Recall Sequence Modeling with Sparse Memory Routing”. In: *arXiv preprint arXiv:2607.25357* (2026). URL: <https://arxiv.org/abs/2607.25357>.
- [102] Alexis Chevalier et al. “Adapting language models to compress contexts”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, pp. 3829–3846. DOI: 10.18653/v1/2023.emnlp-main.232. URL: <https://aclanthology.org/2023.emnlp-main.232/>.
- [103] Jesse Mu, Xiang Li, and Noah Goodman. “Learning to compress prompts with gist tokens”. In: *Advances in Neural Information Processing Systems 36* (2023), pp. 19327–19352. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/3d77c6dcc7f143aa2154e7f4d5e22d68-Abstract-Conference.html.
- [104] Peitian Zhang et al. “Long context compression with activation beacon”. In: *arXiv preprint arXiv:2401.03462* (2024). URL: <https://arxiv.org/abs/2401.03462>.
- [105] Chenlong Deng et al. “UniGist: Towards General and Hardware-aligned Sequence-level Long Context Compression”. In: *arXiv preprint arXiv:2509.15763* (2025). URL: <https://arxiv.org/abs/2509.15763>.
- [106] Aleksandar Petrov et al. “Long context in-context compression by getting to the gist of gisting”. In: *arXiv preprint arXiv:2504.08934* (2025). URL: <https://arxiv.org/abs/2504.08934>.
- [107] Yuzhen Mao, Michael Y. Li, and Emily B. Fox. *Simplified Sparse Attention via Gist Tokens*. 2026. arXiv: 2604.20920 [cs.LG]. URL: <https://arxiv.org/abs/2604.20920>.
- [108] Chia-Yi Hsu et al. “Safe LoRA: The Silver Lining of Reducing Safety Risks when Fine-tuning Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 65072–65094. DOI: 10.52202/079017-2078. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/77baa7c2a3a675823e89131698fd6e19-Abstract-Conference.html.
- [109] Shuang Ao et al. “Safe Pruning LoRA: Robust Distance-Guided Pruning for Safety Alignment in Adaptation of LLMs”. In: *Transactions of the Association for Computational Linguistics 13* (2025), pp. 1474–1487. DOI: 10.1162/tacl.a.44. URL: <https://aclanthology.org/2025.tacl-1.67/>.

- [110] Guanghao Zhou et al. “LSSF: Safety Alignment for Large Language Models through Low-Rank Safety Subspace Fusion”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 30621–30638. DOI: [10.18653/v1/2025.acl-long.1479](https://doi.org/10.18653/v1/2025.acl-long.1479). URL: <https://aclanthology.org/2025.acl-long.1479/>.
- [111] Megh Thakkar et al. “Combining Domain and Alignment Vectors Provides Better Knowledge-Safety Trade-offs in LLMs”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, 2025, pp. 268–277. DOI: [10.18653/v1/2025.acl-short.22](https://doi.org/10.18653/v1/2025.acl-short.22). URL: <https://aclanthology.org/2025.acl-short.22/>.
- [112] Zhengxuan Wu et al. “ReFT: Representation Finetuning for Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 63908–63962. DOI: [10.52202/079017-2041](https://doi.org/10.52202/079017-2041). URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/75008a0fba53bf13b0bb3b7bff986e0e-Abstract-Conference.html.
- [113] Andy Zou et al. *Representation Engineering: A Top-Down Approach to AI Transparency*. 2023. DOI: [10.48550/arXiv.2310.01405](https://doi.org/10.48550/arXiv.2310.01405). arXiv: 2310.01405 [cs.LG]. URL: <https://arxiv.org/abs/2310.01405>.
- [114] Nina Rimsky et al. “Steering Llama 2 via Contrastive Activation Addition”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 15504–15522. DOI: [10.18653/v1/2024.acl-long.828](https://doi.org/10.18653/v1/2024.acl-long.828). URL: <https://aclanthology.org/2024.acl-long.828/>.
- [115] Andy Arditi et al. “Refusal in Language Models Is Mediated by a Single Direction”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 136037–136083. DOI: [10.52202/079017-4322](https://doi.org/10.52202/079017-4322). URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/f545448535dfde4f9786555403ab7c49-Abstract-Conference.html.
- [116] Guobin Shen et al. “Jailbreak Antidote: Runtime Safety–Utility Balance via Sparse Representation Adjustment in Large Language Models”. In: *International Conference on Learning Representations*. 2025. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/36e3f9e6162d597adada4e0e4fce6861-Abstract-Conference.html.
- [117] Bruce W. Lee et al. “Programming Refusal with Conditional Activation Steering”. In: *International Conference on Learning Representations*. 2025. URL: <https://openreview.net/forum?id=0i47wc10sm>.
- [118] Weixiang Zhao et al. “AdaSteer: Your Aligned LLM Is Inherently an Adaptive Jailbreak Defender”. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2025, pp. 24559–24577. DOI: [10.18653/v1/2025.emnlp-main.1248](https://doi.org/10.18653/v1/2025.emnlp-main.1248). URL: <https://aclanthology.org/2025.emnlp-main.1248/>.
- [119] Jiaqi Wu et al. “SafeInt: Shielding Large Language Models from Jailbreak Attacks via Safety-Aware Representation Intervention”. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 8473–8488. DOI: [10.18653/v1/2025.findings-emnlp.450](https://doi.org/10.18653/v1/2025.findings-emnlp.450). URL: <https://aclanthology.org/2025.findings-emnlp.450/>.

- [120] Eric Hanchen Jiang et al. “Mitigating Over-Refusal in Aligned Large Language Models via Inference-Time Activation Energy”. In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 37930–37950. DOI: [10.18653/v1/2026.acl-long.1759](https://doi.org/10.18653/v1/2026.acl-long.1759). URL: <https://aclanthology.org/2026.acl-long.1759/>.
- [121] Andy Zou et al. “Improving Alignment and Robustness with Circuit Breakers”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 83345–83373. DOI: [10.52202/079017-2651](https://doi.org/10.52202/079017-2651). URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97ca7168c2c333df5ea61ece3b3276e1-Abstract-Conference.html.
- [122] Yanrui Du et al. “MoGU: A Framework for Enhancing Safety of LLMs While Preserving Their Usability”. In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024. DOI: [10.52202/079017-2780](https://doi.org/10.52202/079017-2780). URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/9f7f063144103bf6debb09a3f15e00fb-Abstract-Conference.html.
- [123] Zhenglin Lai et al. “SAFEEx: Analyzing Vulnerabilities of MoE-Based LLMs via Stable Safety-Critical Expert Identification”. In: *Advances in Neural Information Processing Systems*. Vol. 38. 2025. DOI: [10.52202/085713-4344](https://doi.org/10.52202/085713-4344). URL: https://proceedings.neurips.cc/paper_files/paper/2025/hash/bd127877149d4965ad834c75a65b3052-Abstract-Conference.html.
- [124] Hugo Touvron et al. “Llama 2: Open Foundation and Fine-Tuned Chat Models”. In: *arXiv preprint arXiv:2307.09288* (2023). URL: <https://arxiv.org/abs/2307.09288>.

A. Training Configurations

This appendix reports the architecture, data, and optimization configurations for the controlled small-scale study, the 4B and 35B-A3B Safin-1 runs, and Safety State specialization.

A.1. Small-Scale MARCH Validation

Table 10 | Training configuration for the controlled small-scale study.

Configuration	Setting
Training regime	From scratch
Training corpus	Long-Data-Collections
Token budget	50B tokens
Sequence length	16,384 tokens
Global batch size	256 packed sequences (4.19M token slots per update)
Optimizer	Fused AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$)
Peak / minimum learning rate	4×10^{-4} / 4×10^{-5}
Learning-rate schedule	WSD: 256-step warmup, 10,473 stable steps, and 1,192 decay steps
Weight decay	0.1
Gradient clipping	Global ℓ_2 norm of 1.0
Precision	BF16 model computation; FP32 optimizer states

Shared architecture. The recurrent models in the primary controlled comparison use a 21-layer fully recurrent backbone with hidden size 1,536, a SwiGLU intermediate size of 4,096, and untied input and output embeddings. All models employ the Llama 2 tokenizer [124] with a vocabulary size of 32,000. GDN, KDA, and GDN2 use six recurrent heads with $d_k = 192$ and $d_v = 384$, together with a bias-free short convolution of kernel size 4.

For MARCH, state anchors are created every $C = 512$ tokens and all causally visible anchors participate in dense routing. The router uses six query heads, one key head, routing dimension $d_r = 64$, and a learned null candidate. The Log-Linear baseline retains at most 15 hierarchical states.

A.2. Large-Scale Safin-1 Training

Backbones and state-routing configuration. We initialize the dense 4B model from Qwen3.5-4B and the MoE model from Qwen3.5-35B-A3B-Instruct. Both backbones use a 3:1 ratio of GDN to full-attention layers. At both scales, 16 selected GDN layers within layers 10–30 (one-based) are equipped with state anchoring and routed historical readout, with anchors created every $C = 512$ text tokens. The router uses 32 query and 32 key heads with $d_r = 128$ per head, selects the Top-4 candidates, and includes a learned null candidate. Historical readout reuses the native GDN query, without a separate historical-read query projection or route positional encoding.

Initialization. The anchor embedding is initialized as $0.5 e_{\text{BOS}}$. Anchor-query and anchor-key projections are initialized from the corresponding pretrained GDN projections after head expansion, and the routing-query projection is copied from the expanded anchor-query projection. Routing RMSNorm parameters are copied from the input layer normalization, while the null projection and bias are initialized to zero. All remaining backbone weights are loaded from the starting checkpoint.

Training protocol. Within each scale and training stage, Qwen3.5 and Safin-1 use the same training data, token budget, and optimization settings. All runs use Megatron distributed Adam with $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay 0.1, and global gradient clipping at 1.0. Model parameters and forward–backward computation use BF16, with FP32 optimizer states and numerically sensitive operations; the 35B-A3B MoE router also remains in FP32. Global token batches count text-token slots and exclude inserted anchor tokens. During SFT, all language-model parameters are updated while the vision tower and aligner remain frozen, and state-routing parameters are inherited from CPT without reinitialization.

Tables 11 and 12 summarize the scale-specific model and training configurations.

Table 11 | Model configurations for the 4B and 35B-A3B Safin-1 models.

Configuration	4B	35B-A3B
CPT initialization	Qwen3.5-4B	Qwen3.5-35B-A3B-Instruct
Decoder layout	32 layers (24 GDN + 8 full-attention)	40 layers (30 GDN + 10 full-attention)
Hidden size	2,560	2,048
GDN state configuration	32 value heads; 16 key heads; $d_k = d_v = 128$	
State-routing layers	16 selected GDN layers within layers 10–30 (one-based)	

Table 12 | Training configurations for continual pretraining and supervised fine-tuning.

Continual Pretraining		
Configuration	4B	35B-A3B
Training data	Intern-S2 pretraining mixture [58]	
Training tokens	50.004B	
Maximum sequence length	32,768 text tokens	
Training steps	5,961	
Global batch	256 sequences (8.389M token slots)	
Peak / minimum learning rate	$5 \times 10^{-5} / 10^{-6}$	
Learning-rate schedule	WSD: 256-step warmup, 5,109 stable steps, and 596 decay steps	
Supervised Fine-Tuning		
Configuration	4B	35B-A3B
Training data	Nemotron-Cascade-2-SFT-Data [59]	
Training tokens	30.199B	
Maximum sequence length	32,768 text tokens	
Training steps	3,600	7,200
Global batch	256 sequences (8.389M token slots)	128 sequences (4.194M token slots)
Peak / minimum learning rate	$10^{-5} / 10^{-6}$	
Learning-rate schedule	5% warmup (180 steps), then cosine decay	5% warmup (360 steps), then cosine decay

A.3. Safety State Specialization

We compare the persistent Safety State with a rank-8 LoRA control under a matched adaptation protocol. Both methods start from the corresponding Safin-1 SFT checkpoint and use the same examples, optimization schedule, batch size, and 37 updates. For Safety State, we freeze the backbone and optimize only the persistent states. For LoRA, we disable Safety State and update only the adapters while retaining dynamic state anchoring and content-routed historical readout.

Table 13 | Shared adaptation configuration for Safety State and the rank-8 LoRA control.

Configuration	Setting
Training data	1,000 STAR-1 harmful and 915 STAR-benign targets
Context conditions	0-, 512-, and 2,048-token benign prefixes; 5,745 examples in total
Starting checkpoint	Corresponding final Safin-1 SFT checkpoint
Training budget	1 epoch; 37 optimizer steps; global batch of 32 packed sequences
Optimizer	Megatron distributed Adam ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$)
Learning-rate schedule	Peak 3×10^{-4} ; minimum 3×10^{-5} ; 10% warmup followed by cosine decay
Regularization	Weight decay 0.01; global gradient clipping 1.0
Maximum sequence length	8,192 tokens
Numerical precision	BF16 model computation and LoRA adapters; FP32 Safety State parameters and optimizer states